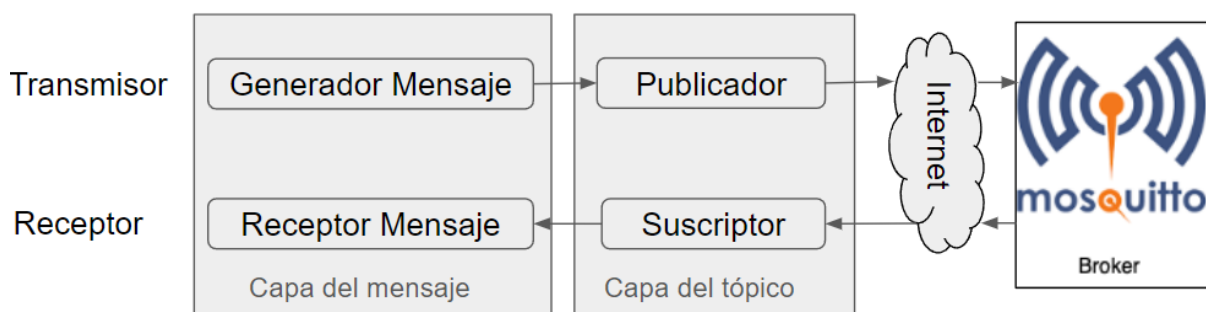


El servidor Mosquito y el servidor Node Red

Daniel Gustavo Rojas Hernández – 2185526
Fabio Andrés Corzo Argüello - 2180383

Tarea 1. Una aplicación Python para acceder remotamente a Mosquitto.

El reto consiste en implementar el sistema que se ilustra en la siguiente figura. Se usará python en un computador remoto para implementar el transmisor (Generador Mensaje y el Publicador), en otro computador remoto se implementará el receptor en python (Receptor Mensaje y Suscriptor). Está claro que en nuestra VM en Azure, tenemos el servidor de Mosquitto.



Paso 1. Debe estar encendida la VM en Azure. No es necesario entrar a la VM porque durante la instalación de Mosquitto, lo programamos para que arranque por defecto, osea que una instancia de mosquitto siempre estará trabajando.

Paso 2. Implementamos el Receptor, mediante Google Colab. [Enlace al código](#). Ponemos a correr ese código para que el suscriptor se quede escuchando a mosquitto.

Paso 3. Implementamos el Transmisor, mediante Google Colab. [Enlace al código](#). Corremos el código, mientras observamos si el mensaje generado en el transmisor llega al receptor.

Para el informe Tarea 1. Un transmisor hecho en python, ubicado en un lugar remoto del mundo, envía mensajes a un receptor similar en otro lugar remoto, usando a nuestro servidor Mosquitto como broker para la distribución de los mensajes.

Explique si se logra el responder positivamente al reto planteado y qué pasa en cada componente del sistema

Se realiza una configuración exitosa del sistema de mensajería gracias a la correcta implementación del código en Python, que puede ejecutarse tanto en Google Colab como en otros entornos de desarrollo, como el IDLE Shell de Python. Los pasos para seguir se detallan a continuación para asegurar una distribución efectiva de mensajes entre el transmisor y el receptor, ambos comunicándose a través de un servidor Mosquitto alojado en una máquina virtual (VM) en Azure. Esta configuración incluye la apertura del puerto

1883 tanto en el firewall del sistema como en las reglas de entrada de Azure, garantizando una conexión adecuada para la comunicación MQTT.

Configuración:

1. Verificación de la VM en Azure

- Es fundamental que la VM en Azure esté encendida y operativa, ya que el servidor Mosquitto se configura para iniciarse automáticamente al encender la VM. Para verificar el estado de Mosquitto, se puede ejecutar `systemctl status mosquitto` en la terminal de la VM, asegurando que esté activo sin necesidad de intervención constante.

2. Implementación del Receptor (Suscriptor) en Python

- **Configuración:** El receptor se suscribe al servidor Mosquitto y se conecta a la IP pública de la VM en el puerto 22. La IP debe ser precisa para que el receptor pueda establecer conexión con el servidor de Azure.
- **Ejecución:** El receptor se ejecuta en un entorno como Google Colab y permanece activo en modo escucha usando `miCliente.loop_forever()`, lo que permite recibir mensajes de manera continua.

3. Implementación del Transmisor (Publicador) en Python

- **Configuración:** El transmisor envía mensajes a un tópico específico (por ejemplo, `MFC_topic`) en el servidor Mosquitto. Es esencial que el tópico del transmisor coincida con el del receptor para que los mensajes se distribuyan correctamente.
- **Pruebas:** Al ejecutar el transmisor, el mensaje debe aparecer en el receptor si la comunicación es exitosa, confirmando que ambos sistemas están correctamente enlazados.

Explicación Técnica del Sistema

- **Mosquitto como Broker MQTT:** Mosquitto gestiona la distribución de mensajes entre el transmisor y el receptor, actuando como intermediario entre ambos. Esta configuración permite simular un sistema de comunicación IoT donde los mensajes se envían y reciben de manera eficiente.
- **Roles del Transmisor y el Receptor:** El transmisor publica mensajes en un tópico, y el receptor, suscrito al mismo, recibe estos mensajes. Esto facilita un flujo de datos en tiempo real entre ambos dispositivos.

Consideraciones Adicionales

- **Conexión Remota:** La configuración permite conectarse al servidor Mosquitto desde entornos remotos como Google Colab, simplificando el proceso de pruebas sin necesidad de instalar MQTT localmente.
- **Sincronización de Tópicos:** Es crucial que el transmisor y el receptor utilicen el mismo tópico para que la comunicación funcione sin inconvenientes.

Esta solución proporciona una estructura robusta para sistemas de comunicación remota y IoT, permitiendo la transferencia eficiente de mensajes mediante un servidor Mosquitto en Azure.

Practica_3_transmisor.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

+ Código + Texto

0s [21]

<ipython-input-21-5c89cf2c54e0>:7: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client()
<MQTTErrorCode.MQTT_ERR_SUCCESS: 0>

Generador mensaje

0s [23]

message = "Hola a todos los que puedan ver este mensaje les habla Daniel Rojas"

0s

Define the topic and message
topic = "MiTopico"
client.publish(topic, message)

<paho.mqtt.client.MQTTMessageInfo at 0x7b11e41c6e80>

Aqui ocurre la desconexion

[]

Disconnect from the server
client.disconnect()

<MQTTErrorCode.MQTT_ERR_NO_CONN: 4>

Practica_3_receptor.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

+ Código + Texto

Haz doble clic (o ingresa) para editar

0s [16]

micliente = mqtt.Client()
micliente.on_connect = mi_callback_de_conexion
micliente.on_message = mi_callback_de_mensajes

micliente.connect("172.161.96.227", 1883)

<ipython-input-16-eefdfd539269>:1: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
micliente = mqtt.Client()
<MQTTErrorCode.MQTT_ERR_SUCCESS: 0>

Poner en loop al cliente para que se quede siempre escuchando

micliente.loop_forever()

... Conectado al broker MQTT
Mensaje recibido: MiTopico Hola a todos los que puedan ver este mensaje les habla Daniel Rojas

```
azureuser@e3tiot: ~
Connection to 172.161.96.227 closed by remote host.
Connection to 172.161.96.227 closed.
PS C:\Users\danie\OneDrive - UNIVERSIDAD INDUSTRIAL DE SANTANDER\Universidad\IoT\API
i .\e3tiot_claves.pem
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1017-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Nov  7 13:59:13 UTC 2024

System load:  0.47          Processes:            124
Usage of /:   6.2% of 28.02GB Users logged in:          0
Memory usage: 4%          IPv4 address for eth0: 10.1.0.4
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Thu Nov  7 13:48:37 2024 from 191.156.245.109
azureuser@e3tiot:~$ |
```

Piense y describa una solución del mundo real que podría ser implementada usando este sistema como punto de partida.

una **red de monitoreo de calidad del aire y tráfico en una ciudad inteligente**. En esta aplicación, se desplegarían sensores de calidad del aire y cámaras de tráfico en varios puntos de la ciudad para obtener datos en tiempo real. Estos datos ayudarían a las autoridades municipales a monitorear la calidad del aire, identificar áreas de congestión vehicular y optimizar la movilidad urbana.

Descripción de la Solución

1. Sensores y Cámaras en Puntos Estratégicos de la Ciudad

- Se colocan sensores de calidad del aire en zonas con alta densidad de tráfico y áreas industriales para medir niveles de CO₂, partículas finas (PM2.5, PM10), y otros contaminantes. Además, se instalan cámaras con sensores de flujo de tráfico en calles y avenidas principales para monitorear la congestión vehicular y la velocidad promedio.
- Cada dispositivo (sensor o cámara) está conectado a un microcontrolador (como un ESP32 o Raspberry Pi) que publica los datos a través de MQTT en un tema específico (por ejemplo, ciudad/aire/calidad, ciudad/trafico), enviándolos al servidor Mosquitto en la nube.

2. Servidor Mosquitto en la Nube

- El servidor Mosquitto, hospedado en una VM de Azure, recibe y centraliza los datos en tiempo real. Esto permite que diferentes sistemas y aplicaciones se suscriban a los temas relevantes y procesen la información sin interferencias ni latencias notables.

3. Tablero de Control para el Municipio

- Un sistema de monitoreo en tiempo real, accesible para las autoridades, muestra la calidad del aire y el estado del tráfico en una plataforma visual.

Este tablero puede incluir alertas en caso de niveles altos de contaminación o congestión crítica, permitiendo que el personal tome medidas inmediatas, como desviar el tráfico o recomendar rutas alternativas a los ciudadanos a través de aplicaciones móviles.

4. Integración con Sistemas de Movilidad Urbana

- En colaboración con aplicaciones de transporte, el sistema de monitoreo de tráfico podría enviar recomendaciones en tiempo real a plataformas como Google Maps o Waze para optimizar las rutas de los conductores y reducir el tiempo en carretera.
- Los datos de calidad del aire también pueden integrarse en sistemas de transporte público, ajustando la frecuencia de los autobuses en áreas de alta contaminación para reducir la emisión de gases y fomentar el uso del transporte público en vez de vehículos privados.

5. Automatización de Alertas y Reportes de Salud Pública

- Cuando la calidad del aire supera ciertos niveles, el sistema podría enviar alertas automáticas a los dispositivos de los ciudadanos (vía una app o mensaje SMS) recomendándoles evitar áreas específicas o permanecer en interiores, particularmente para personas sensibles como niños y adultos mayores.

Tarea 2. Configuración node-RED. Instalación de complementos mínimos.

Paso 1: Asegúrate de tener la VM encendida. Entra a ella mediante una terminal SSH ([Enlace a explicación en el manual Azure sobre como abrir la VM por SSH](#)). Pon a correr node-RED mediante el comando:

```
node-red -v
```

Paso 2: Abrir Node-RED en un navegador

Comprueba que, sin tocar la máquina, el navegador de internet permite el acceso remoto a Node-RED. En el navegador escribe la IP pública seguida de “: 1880”, por ejemplo, si la IP es “54.236.56.149” deberías escribir:

<http://54.236.56.149:1880>

Paso 3. Instalar complementos mínimos de Node-RED

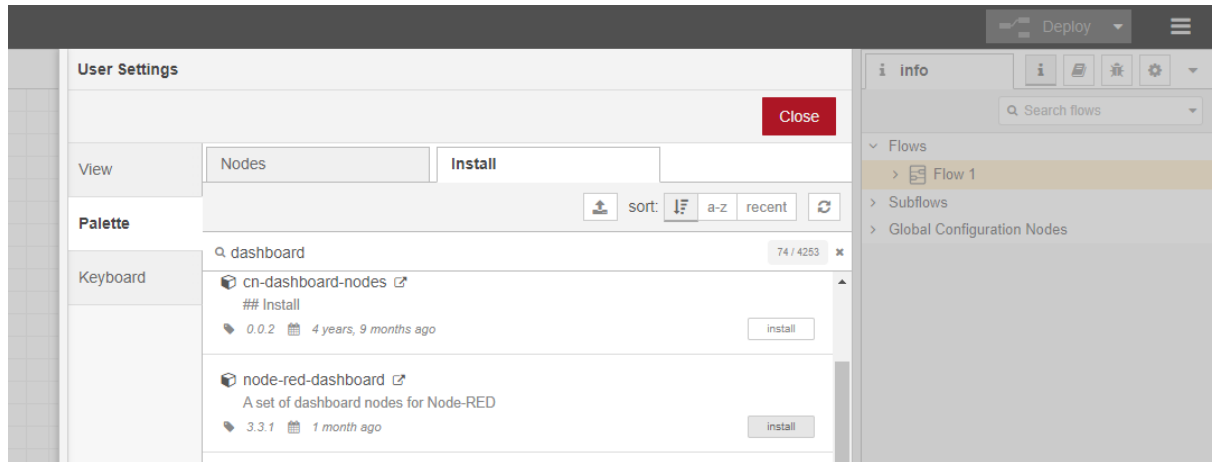
El proceso para cualquier instalación es el siguiente:



> Manage Palette > Install > en el buscador escribe el nombre de lo que desea instalar y oprime enter > selecciona de la lista el producto que buscas y oprime el botón “Install”> Install

- Sigue el proceso señalado para instalar: **node-red-dashboard**

Es un módulo adicional para Node-RED que permite crear interfaces de usuario (dashboards) interactivas y personalizadas. Estas interfaces pueden incluir gráficos, botones, controles deslizantes, medidores, e indicadores que te permiten visualizar y controlar los flujos de datos en tiempo real. Este complemento es especialmente útil para aplicaciones de Internet de las Cosas (IoT), donde puedes necesitar una forma intuitiva de monitorizar y manejar dispositivos y sensores conectados.



- de manera similar instalar: **node-red-contrib-ui-led**
Permite agregar indicadores LED virtuales a tus dashboards creados con el "node-red-dashboard". Estos indicadores LED son útiles para mostrar el estado de diferentes procesos, dispositivos o señales de manera visual e intuitiva. Por ejemplo, puedes utilizar un LED verde para indicar que un sistema está operativo y un LED rojo para señalar un error o una condición de alerta.
- instalar también: **node-red-mysql**. Nota: en 2023-2 vimos que ya no existe node-red-mysql sino **node-red-mysql-r2**. Todo indica que hace lo mismo- permite que un nodo ejecute solicitudes MySQL sobre una base de datos dada.

Paso 4: Llene la siguiente tabla para el informe.

Para el informe. Tarea 2: Evidencias de la instalación de complementos Node-RED
Evidencias del dashboard agregado. Tome una captura del menú lateral izquierdo, de manera que sea posible ver que existe una sección que despliega de nodos llamada "dashboard"

User Settings

Close

View

Nodes

Install

filter nodes

Palette

Keyboard

Environment

dashboard-evi

1.0.2

> 21 nodes

in use

node-red

3.1.9

> 50 nodes

in use

node-red-contrib-ui-led

0.4.11

> 1 node

remove

disable all

node-red-contrib-ui-media

1.2.0

> 1 node

remove

disable all

node-red-mysql-r2

1.3.0

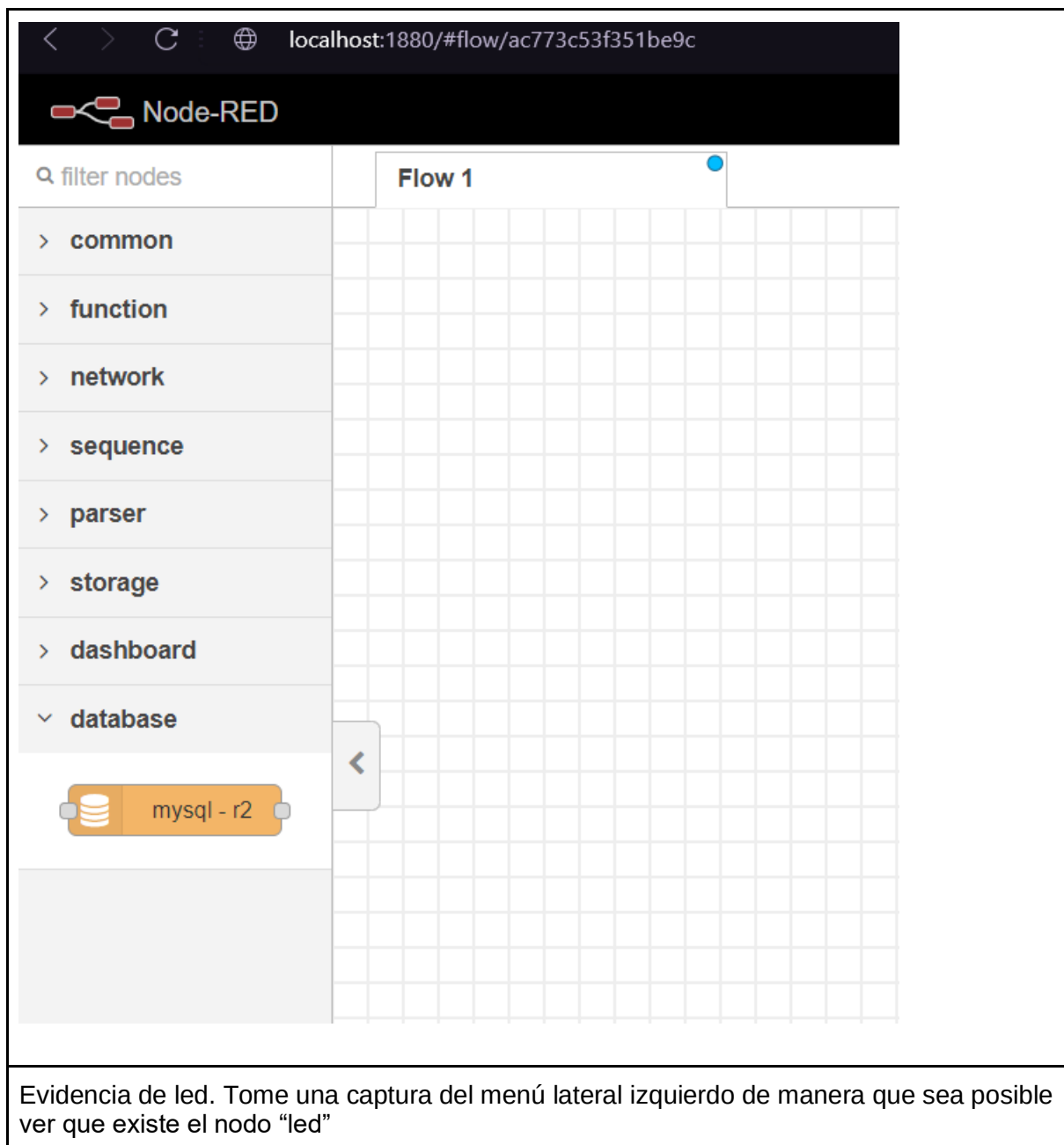
> 1 node

remove

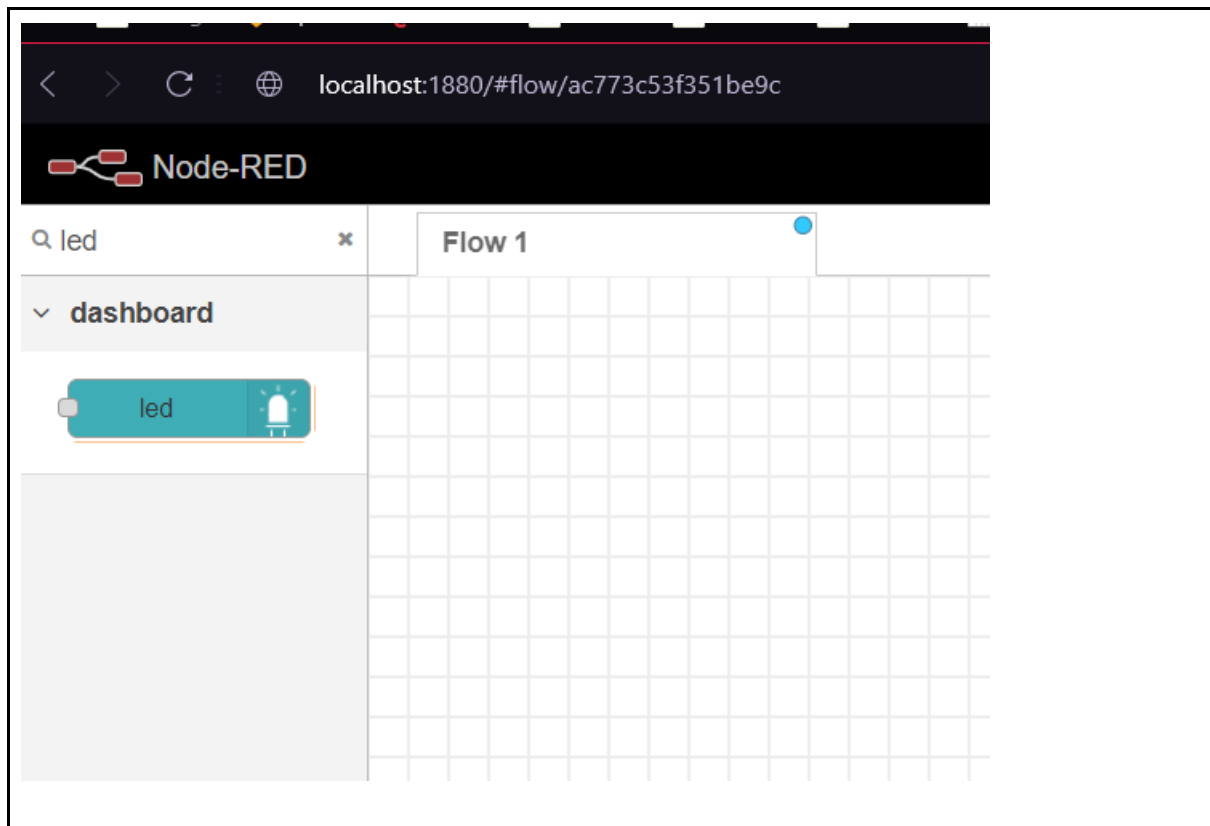
disable all



Evidencia de mysql - r2. Tome una captura del menú lateral izquierdo de manera que sea posible ver que existe una sección que despliega de nodos llamada "database" que tiene el nodo "mysql - r2"



Evidencia de led. Tome una captura del menú lateral izquierdo de manera que sea posible ver que existe el nodo "led"



Tarea 3. Pruebas de funcionamiento de Node Red

Node-RED usa el término “nodo” para referirse a lo que nosotros llamaríamos, de manera más natural, bloques. A veces le llamaremos bloques, pero recuerda que el término oficial es “nodo”. También es importante recordar que el conjunto de varios nodos enlazados, también se llama “nodo”, es muy similar a la programación orientada a objetos, donde varios objetos pueden componer a un nuevo objeto. Nosotros, para evitar confusiones llamaremos flujo-nodo a los nodos enlazados o flujogramas de nodos.

Paso 1. Implementación y configuración de un sistema IoT dentro de la misma VM

- Sistema a implementar:

La Figura siguiente contiene los elementos que implementaremos en esta prueba. Lo que indica es que hay que crear dos soluciones: una para el transmisor y otra para el receptor. El profesor ha ideado la manera de representar las dos cosas como un solo sistema de comunicaciones en un modelo de capas.

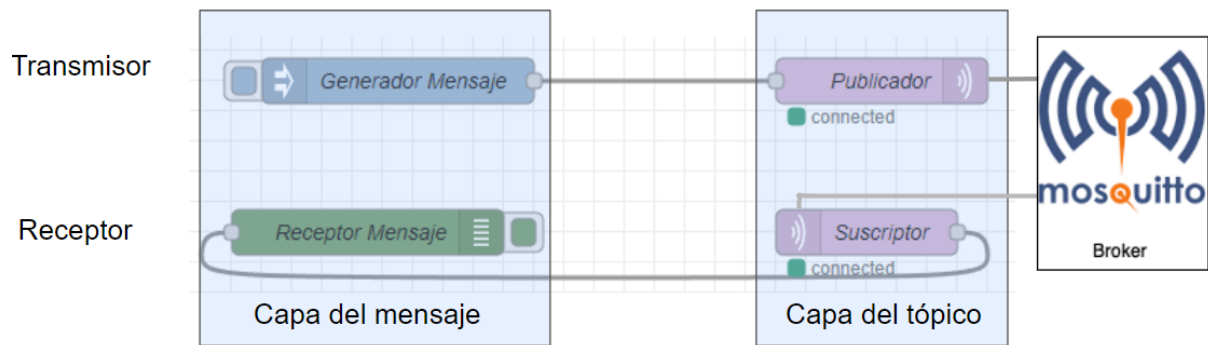


Figura 2.

- bloques (nodos) a usar:

Busque en el menú lateral los siguientes bloques, arrástrelos al área de trabajo del flujo-nodo actual.

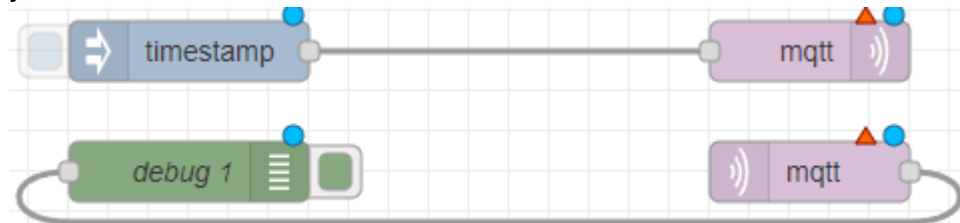
mqtt out: Representa al publicador "MiTopico"

mqtt in: Representa al suscriptor "MiTopico"

inject: Representa el Generador Mensaje

debug: Representa el Receptor Mensaje

y realice las conexiones así:



- Configuraciones

Publicador (nodo mqtt out)

Server

localhost:1883

Topic

MiTopico

QoS

0

Retain

Name

Publicador

Suscriptor (nodo mqtt in)

Server

localhost:1883

Action

Subscribe to single topic

Topic

MiTopico

QoS

2

 Output

auto-detect (parsed JSON object, string or buf

 Name

Suscriptor

Generador Mensaje (nodo inject)

Name

Generador Mensaje

msg. payload

=

a_z

hola, soy un bloquecillo de node red.

msg. topic

=

a_z

Receptor Mensaje (nodo debug)

Edit debug node

Delete

Cancel

Done

Properties

Output

complete msg object

To

☒ debug window

☐ system console

☐ node status (32 characters)

Name

Receptor Mensaje

Paso 2. Funcionamiento

- Se oprime Deploy para que arranque

- Se oprime el botón izquierdo  del nodo “Generador Mensaje” cada vez que se desee enviar un nuevo mensaje
- Observa que el panel derecho, si se oprime , corresponde a la ventana de visualización del nodo debug

La siguiente figura es como aparecen los resultados

```
2/28/2023, 7:06:40 PM node: Receptor Mensaje
MiTopico : msg : Object
  ▶ { topic: "MiTopico", payload:
    "hello, soy un bloquesillo de n...",
    qos: 0, retain: false, _topic:
    "MiTopico" ... }
```

Que también, si se oprime ▶, veremos que se trata de los datos emitidos, pero organizados en formato JSON

```
2/28/2023, 7:06:40 PM node: Receptor Mensaje
MiTopico : msg : Object
  ▼ object
    topic: "MiTopico"
    payload: "hello, soy un bloquesillo
    de node red."
    qos: 0
    retain: false
    _topic: "MiTopico"
    _msgid: "7d863bebb5e0a8f5"
```

Paso 3. Conquista del Receptor Mensaje (nodo debug)

- Configurar el nodo para extraer del JSON y presentar en pantalla solo los datos que corresponden a valores. Entonces lo que interesa es msg.payload y la configuración es:

 Output ▼ msg. payload

Oprima Done > Deploy >  en el “Generador Mensaje”. El resultado debe ser

```
2/28/2023, 7:27:17 PM node: Receptor Mensaje
MiTopico : msg.payload : string[38]
"hello, soy un bloquesillo de node
red."
```

- De manera similar, configure el nodo para que muestre solo el tópico

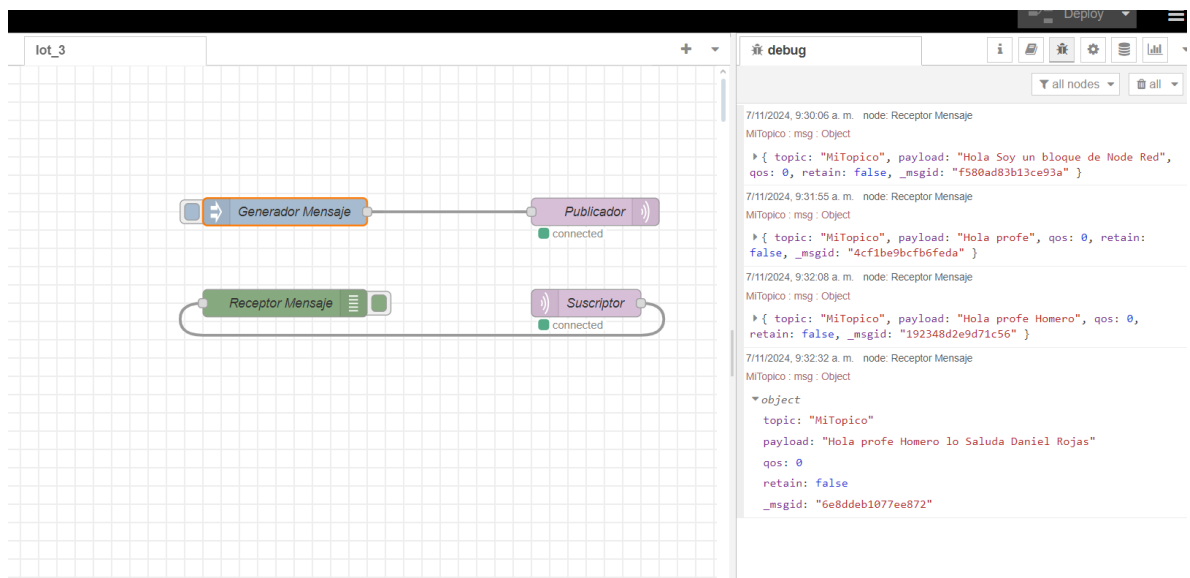
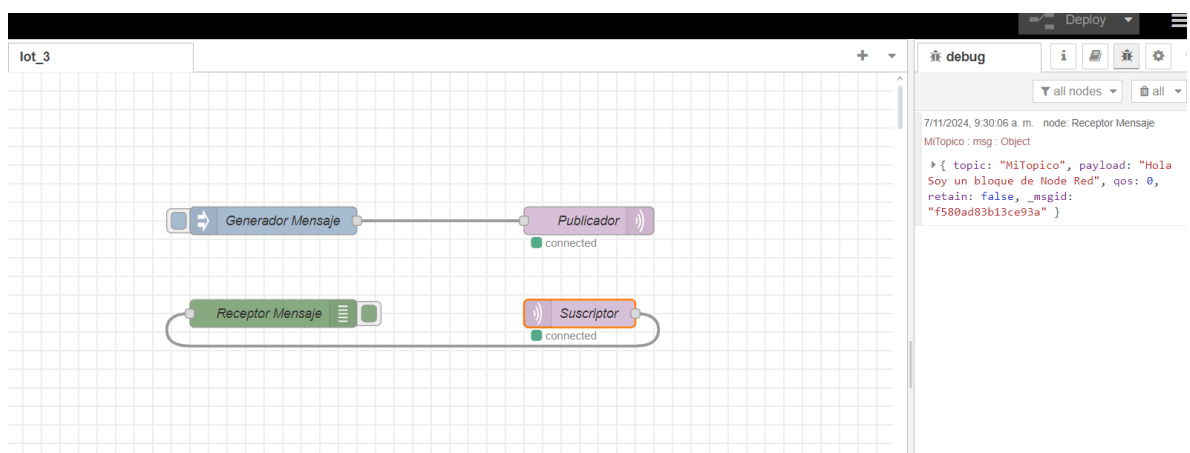
Paso 4. Conquista total de los nodos usados:

Comprueba lo que ocurre cuando:

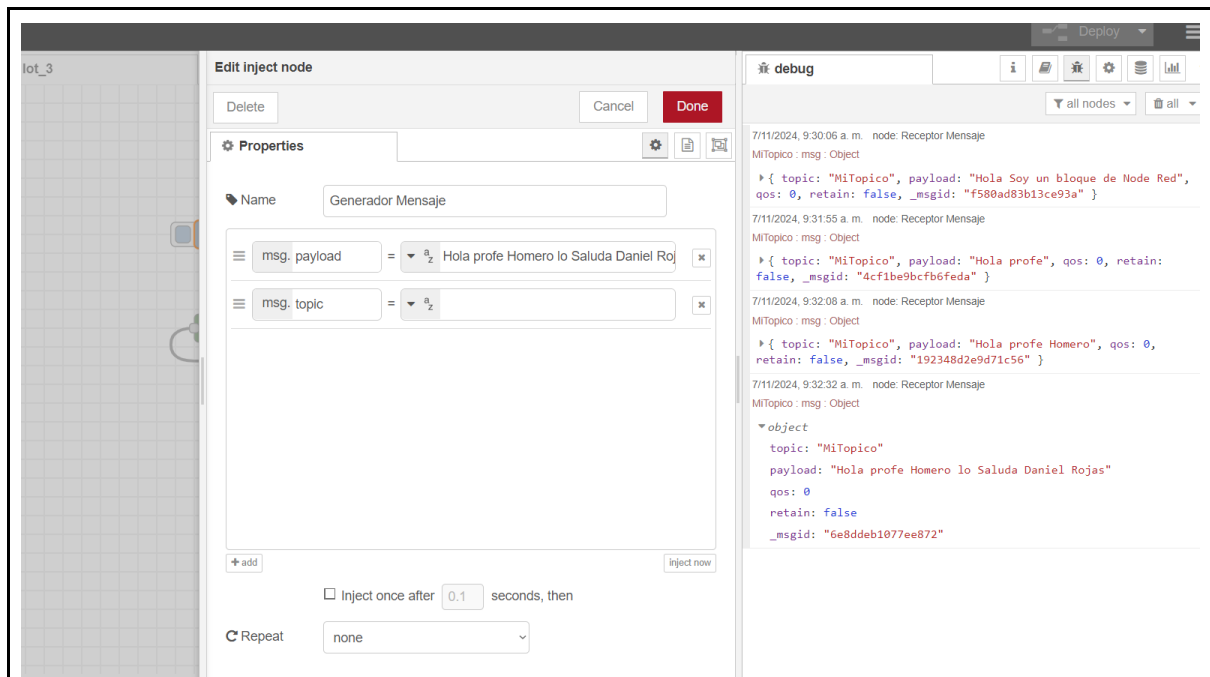
- se activa o desactiva el botón que tiene a la derecha el bloque (nodo) debug (Receptor Mensaje).
- se oprime el botón a la izquierda del bloque (nodo) inject (Generador Mensaje).
- pruebe configurar el inject (Generador Mensaje) para que genere mensajes cada segundo.
- compruebe que significa y en qué casos debería aparecer la señal  **connected** debajo de algunos bloques

Para el informe. Tarea 3. Tabla de resultados de primera implementación en Node-RED

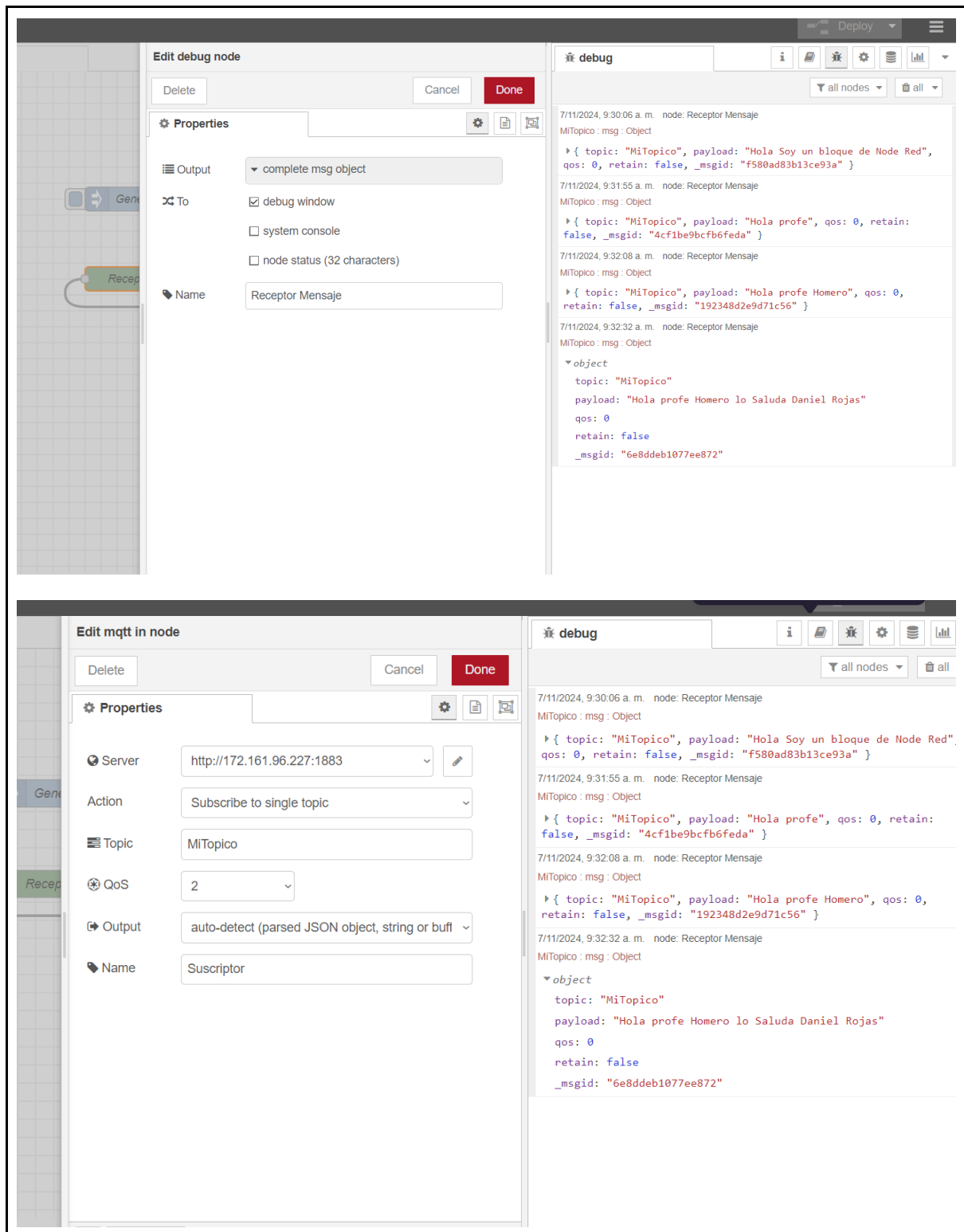
Pegue aquí una captura de pantalla del flujo-nodo implementado



De acuerdo al paso 2, muestre un pantallazo en el que aparece la configuración del “Generador Mensaje” y el resultado que entrega el “Receptor Mensaje”. Nota: use mensajes originales, propios, diferente a los de esta guía o de otros compañeros.

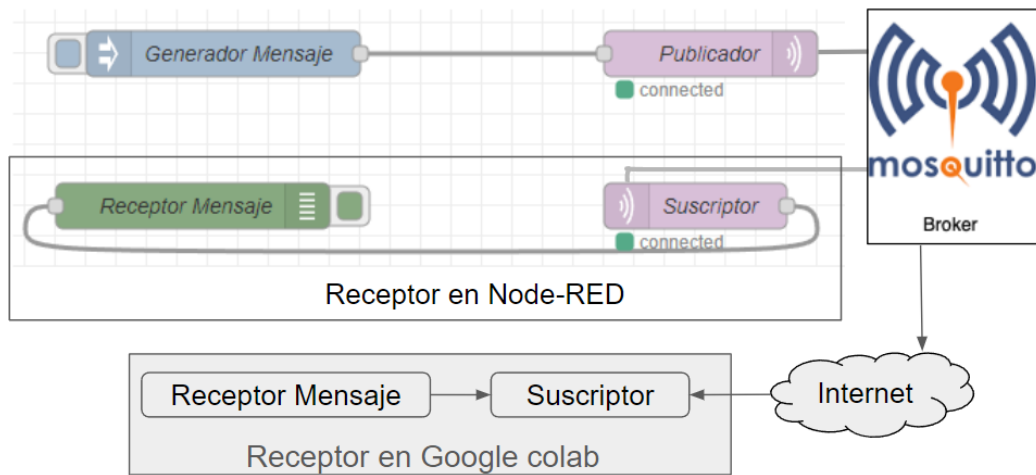


De acuerdo al paso 3, muestre un pantallazo de la configuración del “Receptor Mensaje” para obtener solo el tópico. También muestre el resultado que entrega el “Receptor Mensaje”



Tarea 4.

Usando todo lo aprendido, implemente una solución como la que se presenta en la figura siguiente



La idea es tan simple como comprobar que el mensaje que se genera, no solo llegue al “Receptor Mensaje” de Node-RED sino también, al mismo tiempo al implementado en Python con Google Colab.

Para el informe. Tarea 4.

Muestre abajo el resultado de la implementación del “Suscriptor” remoto del mensaje.

```

Practica_3_transmisor.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

+ Código + Texto

[27] client.connect(server_address, server_port)
0s
<ipython-input-27-5c89cf2c54e0>:7: DeprecationWarning: Callback API version 1 is deprecated, update to latest
client = mqtt.Client()
<MQTTErrorCode.MQTT_ERR_SUCCESS: 0>

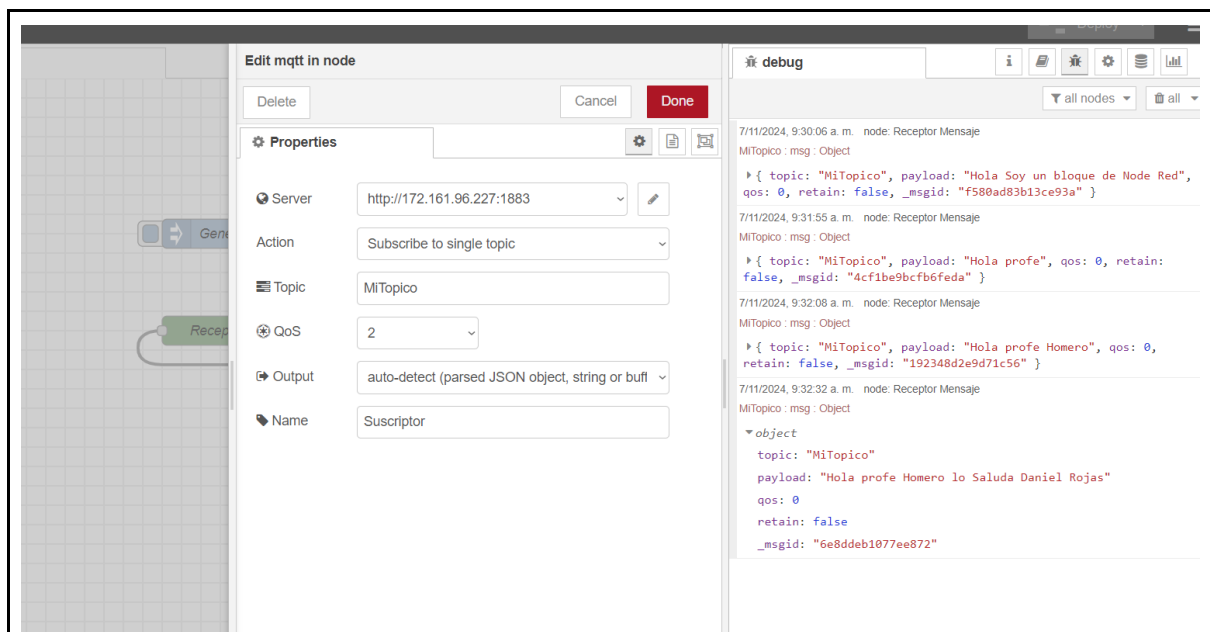
v Generador mensaje

[33] message = "Hola a todos los que puedan ver este mensaje les habla Daniel Rojas m"
0s

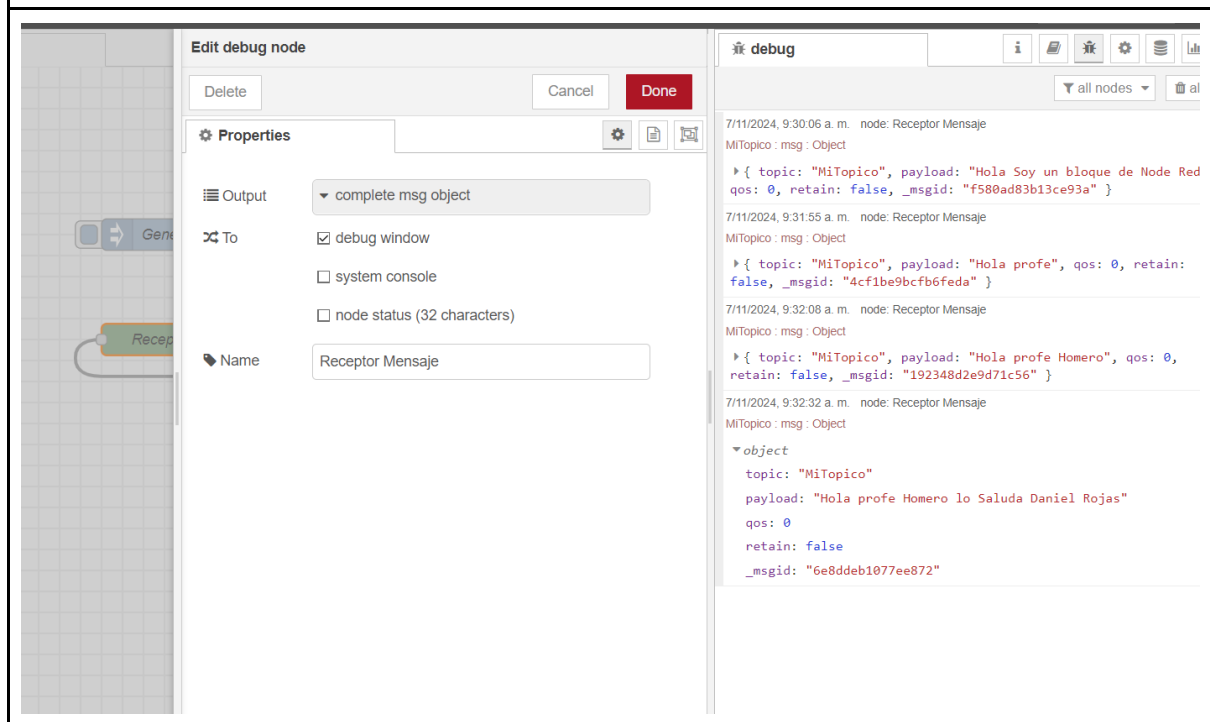
# Define the topic and message
topic = "MiTopico"
client.publish(topic, message)
<paho.mqtt.client.MQTTMessageInfo at 0x7b11e41b1cb0>

v Aqui ocurre la desconexion

[ ] # Disconnect from the server
client.disconnect()
<MQTTErrorCode.MQTT_ERR_NO_CONN: 4>
  
```



Muestre abajo el resultado de la implementación del “Receptor Mensaje” del mensaje.



Practica_3_receptor.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

+ Código + Texto

Haz doble clic (o ingresa) para editar

[19]

micliente = mqtt.Client()
micliente.on_connect = mi_callback_de_conexion
micliente.on_message = mi_callback_de_mensajes

micliente.connect("172.161.96.227", 1883)

<ipython-input-19-eefdfd539269>:1: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
micliente = mqtt.Client()
<MQTTErrorCode.MQTT_ERR_SUCCESS: 0>

▼ Poner en loop al cliente para que se quede siempre escuchando

micliente.loop_forever()

... Conectado al broker MQTT
Mensaje recibido: MiTopico Hola a todos los que puedan ver este mensaje les habla Daniel Rojas
Mensaje recibido: MiTopico Hola Soy un bloque de Node Red
Mensaje recibido: MiTopico Hola profe
Mensaje recibido: MiTopico Hola profe Homero
Mensaje recibido: MiTopico Hola profe Homero lo Saluda Daniel Rojas

Muestre una comparación entre el mensaje en el “generador del mensaje” y lo que se obtiene en el Receptor en Google Colab

Practica_3_receptor.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

+ Código + Texto

Haz doble clic (o ingresa) para editar

[19]

micliente = mqtt.Client()
micliente.on_connect = mi_callback_de_conexion
micliente.on_message = mi_callback_de_mensajes

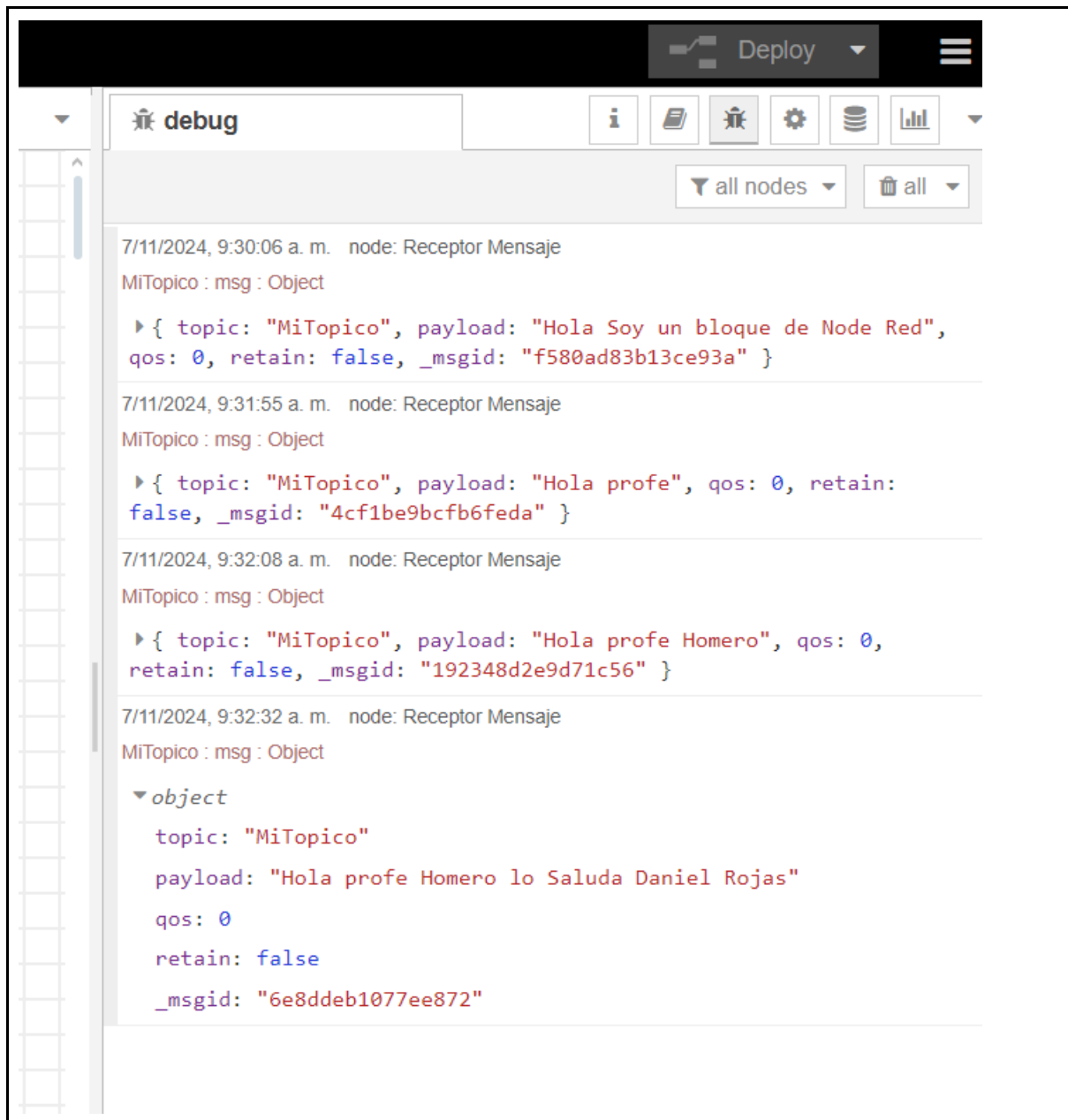
micliente.connect("172.161.96.227", 1883)

<ipython-input-19-eefdfd539269>:1: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
micliente = mqtt.Client()
<MQTTErrorCode.MQTT_ERR_SUCCESS: 0>

▼ Poner en loop al cliente para que se quede siempre escuchando

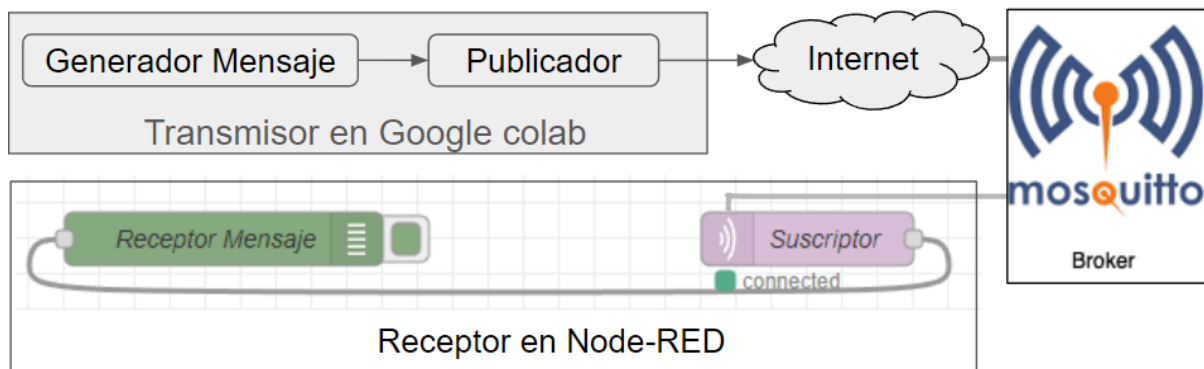
micliente.loop_forever()

... Conectado al broker MQTT
Mensaje recibido: MiTopico Hola a todos los que puedan ver este mensaje les habla Daniel Rojas
Mensaje recibido: MiTopico Hola Soy un bloque de Node Red
Mensaje recibido: MiTopico Hola profe
Mensaje recibido: MiTopico Hola profe Homero
Mensaje recibido: MiTopico Hola profe Homero lo Saluda Daniel Rojas



Tarea 5.

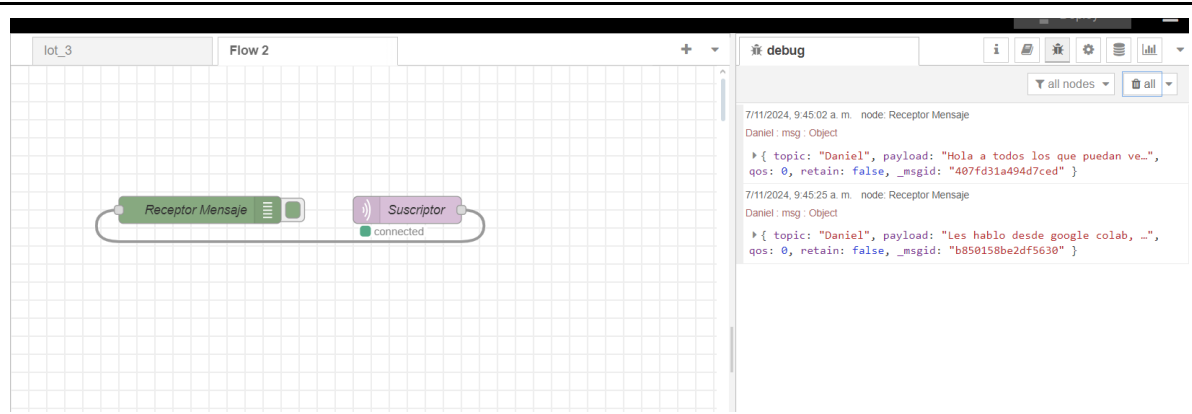
Usando todo lo aprendido, implemente una solución como la que se presenta en la figura siguiente



Es decir, el publicar está en un lugar remoto, implementado en python, con google colab.
 Nota: en lugar de google colab, puede usar una tarjeta ESP32 o una Raspberry PI o un programa en Python o cualquier otro lenguaje de programación.

Para el informe. Tarea 5.

Muestre abajo el resultado de la implementación del generador del mensaje.



Muestre abajo el resultado de la implementación del publicador del mensaje.

Practica_3_transmisor.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios

+ Código + Texto

✓ 0 s

[36]

client.connect(server_address, server_port)

<ipython-input-36-5c89cf2c54e0>:7: DeprecationWarning: Callback API version 1 is deprecated, update
client = mqtt.Client()
<MQTTErrorCode.MQTT_ERR_SUCCESS: 0>

Generador mensaje

✓ 0 s

[40]

message = "Les hablo desde google colab, Les Saluda Daniel"

✓ 0 s

Define the topic and message
topic = "Daniel"
client.publish(topic, message)

<paho.mqtt.client.MQTTMessageInfo at 0x7b11e41b2430>

Aqui ocurre la desconexion

✓ 0 s

[35]

Disconnect from the server
client.disconnect()

<MQTTErrorCode.MQTT_ERR_NO_CONN: 4>

Muestre una comparación entre el mensaje en el “generador del mensaje” y lo que se obtiene en Node-RED

Generador mensaje

✓ 0 s

[40]

message = "Les hablo desde google colab, Les Saluda Daniel"

✓ 0 s

Define the topic and message
topic = "Daniel"
client.publish(topic, message)

<paho.mqtt.client.MQTTMessageInfo at 0x7b11e41b2430>

Aqui ocurre la desconexion

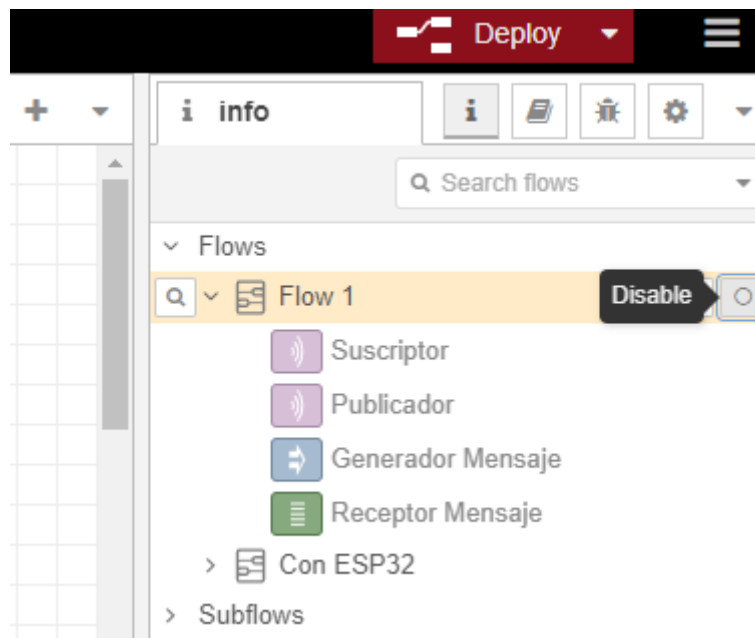


Tarea 6. Conquista del sistema de manejo de flujo-nodos y archivos

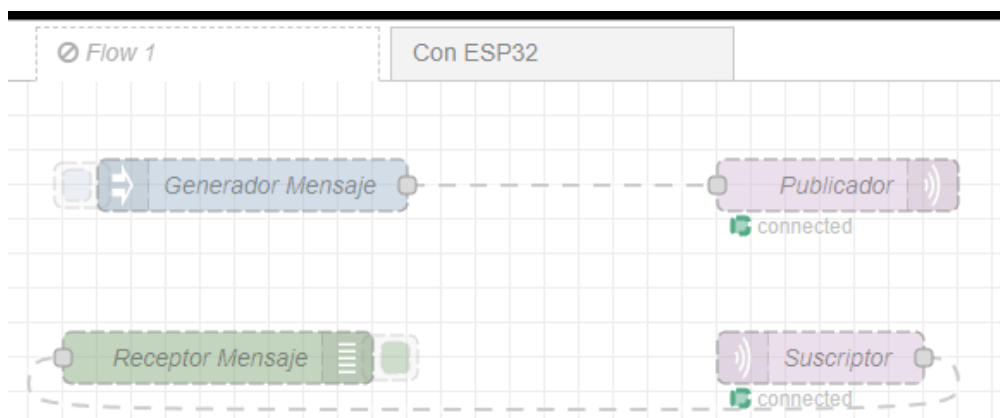
Paso 1. Probar lo más básico

- Analiza si se conservan los cambios de un flujo-nodo, cuando no oprimes Deploy. Para esto, haz unos cambios al flujo-nodo, copia la URL de tu trabajo, pasa a otra ventana del navegador, pega la URL y mira si observas los nuevos cambios o solo lo que había antes de un comando Deploy.
- Descubra cómo cambiarle el nombre a los nodos (flujo-nodos)
- pruebe ocultar un flujo-nodo y luego hacerlo aparecer nuevamente
- Pruebe desactivar/activar uno de los flujo-nodos así:

Menu lateral derecho >  seleccionas en el panel izquierdo el flujograma > disable



El resultado es que el flujograma aparece punteado



Paso 2. Aprender a guardar de manera segura los trabajos hechos en Node-RED

Supongamos que tienes un flujograma que no vas a usar en el momento, pero que deseas conservar para usos futuros. ¿Cómo lo guardarías? ¿Cómo llamarías un flujograma guardado?. Responder estas preguntas es muy importante ya que tenemos amargas experiencias de principiantes que han realizado un fuerte trabajo, pero luego, por alguna razón lo pierden todo.

Realiza la siguiente práctica

- Guarde en su computador el flujograma usado. Menú principal  > Export > Seleccionas el flujograma de interés > Download. Nota: es más profesional guardarlo en un repositorio como GitHub.
- Crea un flujo nodo de pruebas. Borra del Node-RED el flujo nodo (OJO porque estas instrucciones borran sin advertir el flujograma donde estés parado): Te paras en el flujograma de interés > Menú principal  > Flows > Delete

- Recupere en Node-RED el flujograma a partir del archivo guardado en el computador
Menú principal  > Import > New flow > Select a file to import > seleccionas el archivo de interés (observa que existe la opción de subir un archivo de tu computador (Clipboard), de tu VM (localhost), ejemplos que Node-RED ofrecer (Examples))> primer Import > Import copy

Paso 3. Conquista de Menús de consulta

- Compruebe qué se aprecia con: Menú lateral derecho > 
- Compruebe qué se aprecia con: Menú lateral derecho > 
- Compruebe si las ayudas se adaptan al bloque que se selecciona: Resalte uno de los nodos del flujo-nodo > Menú lateral derecho > 
- Crea un nuevo flujo-nodo sin modificar el anterior, ponlo a correr y analiza si los dos flujo-nodos quedan corriendo o solo uno
- Compruebe la posibilidad de filtrar datos con  >  por ejemplo para mostrar sólo datos de un flujo-nodo o de varios.

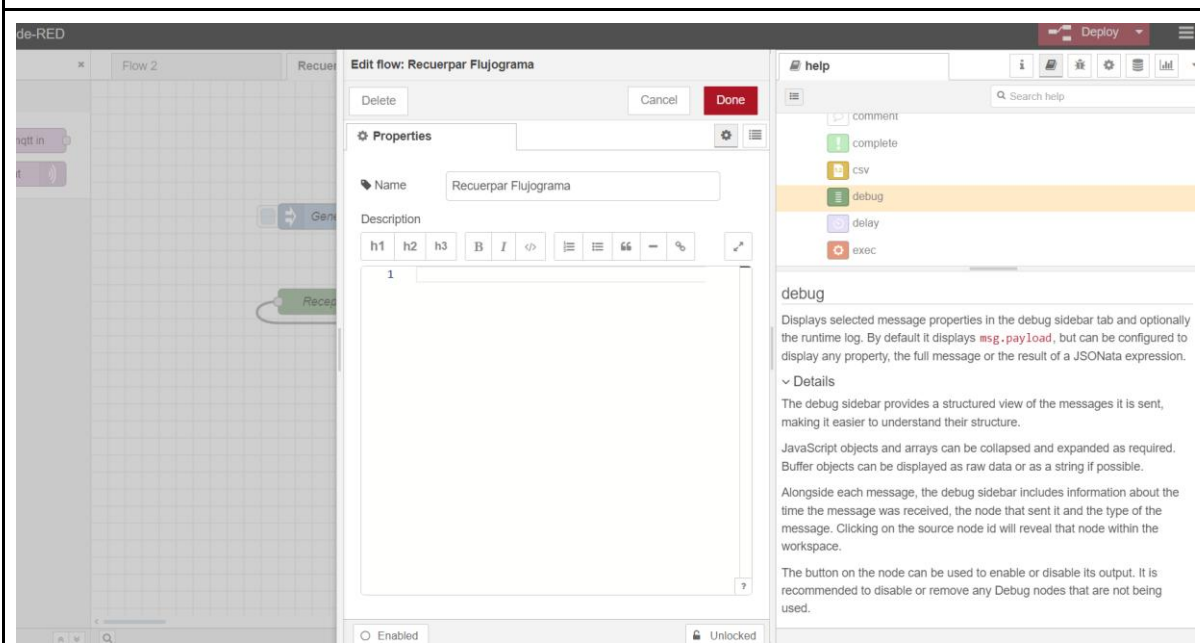
Paso 4. Otras conquistas

- analice si otras personas puede editar su flujo-nodo, desde otro navegador

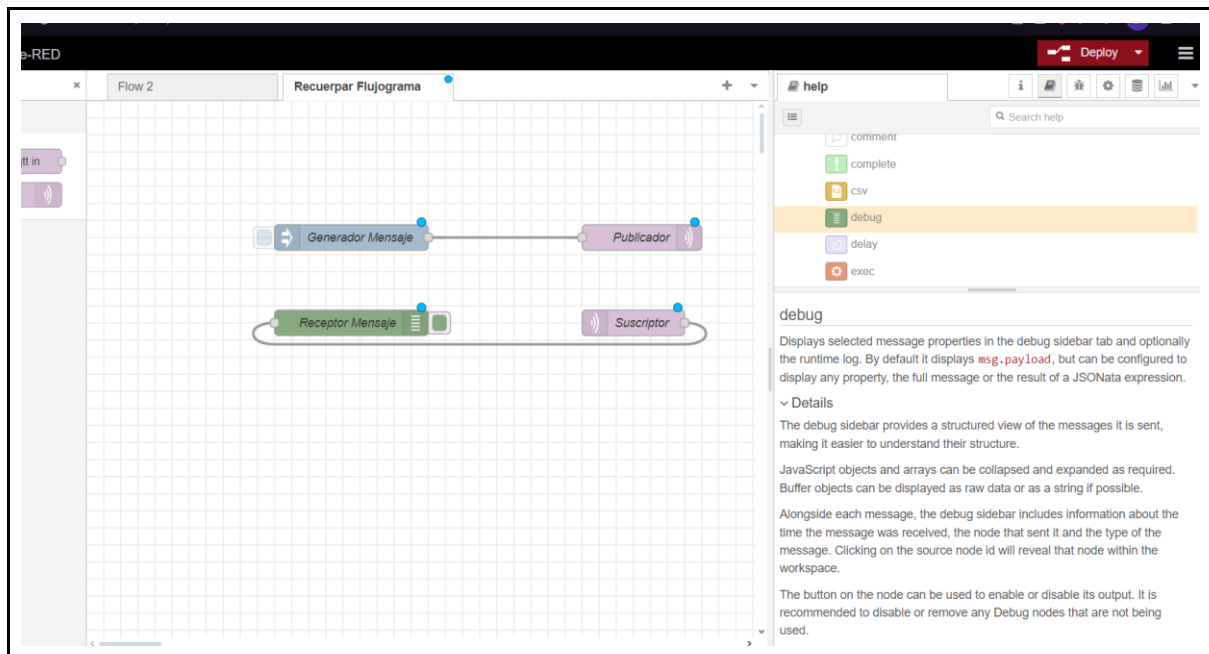
Paso 5. Llene la tabla que va como informe

Para el informe. Tarea 6. Tabla de resultados de la conquista de herramientas de Node-RED

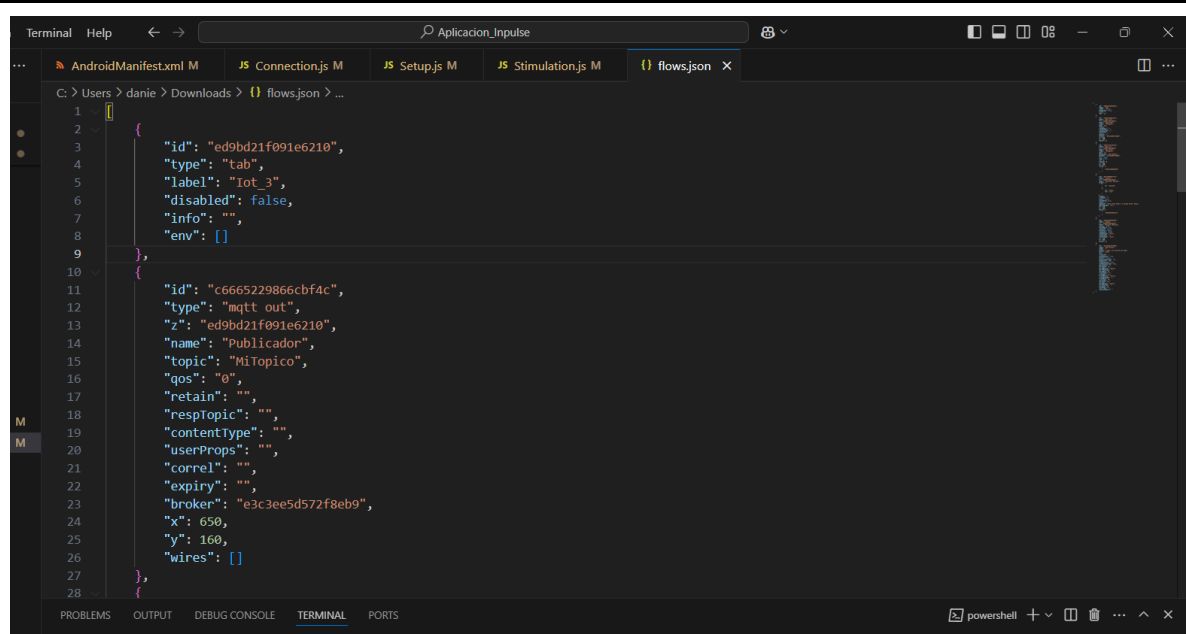
Captura de pantalla que demuestra que has podido cambiarle el nombre a un flujo-nodo



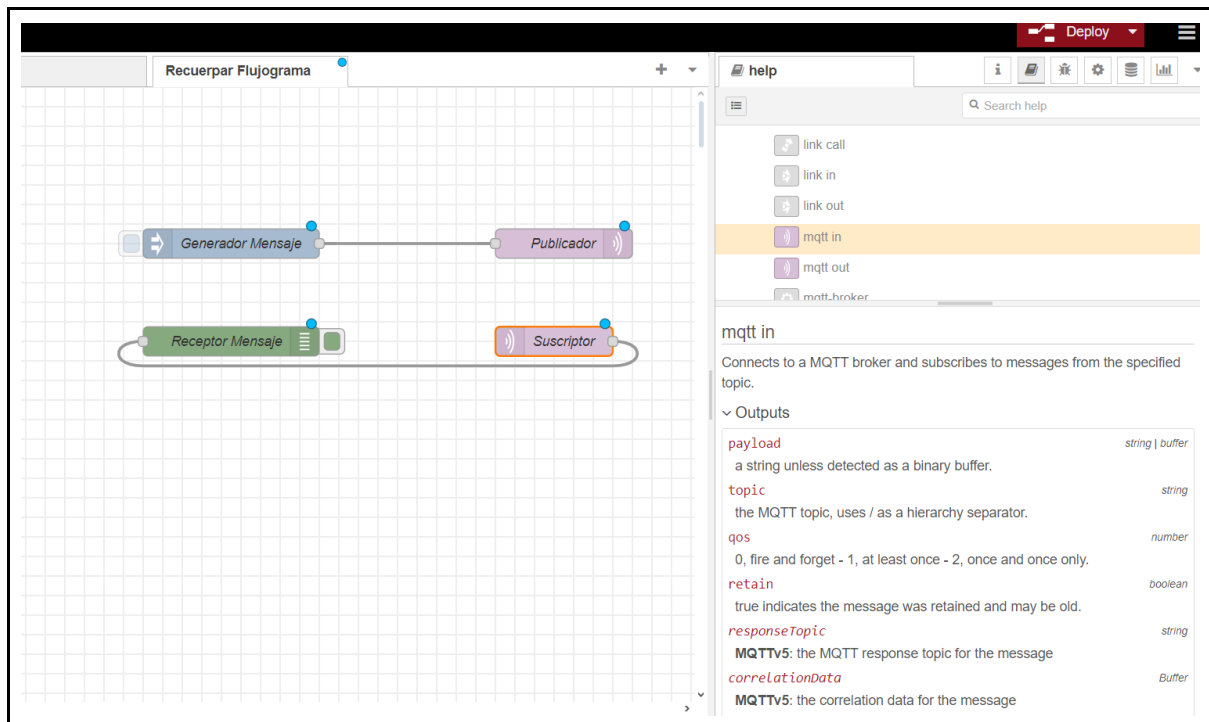
Captura de pantalla que demuestra que has podido desactivar un flujo-nodo



Pantalla que muestra como es el archivo de un flujo-nodo cuando lo has bajado a tu computador



Muestra un pantallazo para demostrar que puedes ver las ayudas de un nodo seleccionado



Explique si, con la configuración que tiene actualmente Node-RED, otros usuarios podrían, desde otro navegador modificar tu flujo-nodo.

Con la configuración actual de Node-RED, es posible que otros usuarios puedan modificar el flujo-nodo desde otro navegador, pero esto dependerá de cómo esté configurado el acceso y la seguridad del entorno. Si Node-RED se está ejecutando en una máquina local o en un servidor con acceso público, es necesario tomar en cuenta ciertos factores:

1. **Autenticación y Permisos:** Si la instancia de Node-RED no está protegida con un usuario y contraseña, cualquier persona que conozca la dirección IP y el puerto de acceso podría modificar el flujo-nodo al ingresar desde otro navegador. Para evitar esto, Node-RED permite habilitar la autenticación en el archivo de configuración, lo cual limita el acceso a usuarios autorizados.
2. **Restricciones de Red:** Si Node-RED está ejecutándose en una red privada o detrás de un firewall, el acceso estará restringido a dispositivos dentro de esa red. Sin embargo, si la aplicación está configurada para permitir el acceso público, otros usuarios podrán acceder y modificar los flujos si conocen la dirección de acceso.
3. **Certificados SSL y HTTPS:** Node-RED también permite configurar un certificado SSL para habilitar el acceso seguro a través de HTTPS. Esto no solo protege la conexión, sino que también ayuda a restringir el acceso mediante la verificación de identidad, reduciendo el riesgo de accesos no autorizados.
4. **Permisos de Acceso en la Plataforma de Hospedaje:** Si Node-RED está alojado en una plataforma como una máquina virtual en la nube, la configuración de permisos de acceso y reglas de seguridad en la plataforma también influirá en quién puede acceder. Por ejemplo, en Azure o AWS, se pueden definir reglas de entrada y salida para el puerto en el que se ejecuta Node-RED, limitando el acceso a direcciones IP específicas.